

CODE, SOFTWARE USED, AND INSTRUCTIONS TO EXECUTE THE SYSTEM

1. Introduction

This document provides a detailed overview of the software, tools, libraries, and execution steps used in the development of the AI-powered brain tumor detection and segmentation system. The system integrates deep learning (Mask R-CNN), computer vision techniques, and a Flask-based web application to deliver real-time segmentation results along with AI-generated insights.

2. Software and Technologies Used

2.1 Programming Languages

- **Python** – Core backend development, model training, and inference
- **JavaScript** – Frontend interactivity and validation
- **HTML & CSS** – UI design and layout

2.2 Frameworks and Libraries

- **Deep Learning Framework:** PyTorch (for Mask R-CNN implementation)
- **Computer Vision:** OpenCV (image preprocessing and visualization)
- **Model Architecture:** Mask R-CNN (instance segmentation)
- **Backend Framework:** Flask (API handling and server logic)
- **Frontend Technologies:** HTML, CSS, JavaScript
- **AI Integration:** Gemini API (for generating textual insights)
- **Data Handling:** NumPy, Pandas
- **Visualization:** Matplotlib

2.3 Tools and Platforms

- **Google Colab / Jupyter Notebook** – Model training and experimentation
- **VS Code / PyCharm** – Development environment
- **Git & GitHub** – Version control and collaboration
- **SQLite** – Database for storing user credentials and prediction history

3. System Requirements

3.1 Hardware Requirements

- Minimum 8 GB RAM (16 GB recommended)
- GPU (NVIDIA CUDA-enabled preferred for training)
- Processor: Intel i5 / Ryzen 5 or higher

3.2 Software Requirements

- Python 3.8 or above
- pip (Python package manager)
- Web browser (Google Chrome recommended)
- CUDA toolkit (optional, for GPU acceleration)

4. Installation and Setup

- Step 1: Clone the Repository

```
git clone <repository-link>
cd brain-tumor-detection
```

- Step 2: Create and Activate Virtual Environment

```
python -m venv venv
```

Activate environment:

- Windows:

```
venv\Scripts\activate
```

- Linux/Mac:

```
source venv/bin/activate
```

- Step 3: Install Dependencies

```
pip install -r requirements.txt
```

- Step 4: Install Additional Required Libraries (if not included)

```
pip install torch torchvision opencv-python flask numpy pandas matplotlib pillow
```

5. Dataset Preparation

- Download MRI dataset (e.g., from Kaggle or medical repositories)

- Organize dataset into folders:

```
dataset/
```

```
|— train
```

```
|— val/
```

```
|— test/
```

- Ensure proper annotations for tumor regions (bounding boxes and masks if required)
- Perform preprocessing:
 - Image resizing
 - Normalization
 - Data augmentation (rotation, flipping, scaling)

6. Model Execution

6.1 Training the Model

`python train.py`

- Trains Mask R-CNN on MRI dataset
- Saves trained weights in `/models` directory

6.2 Model Evaluation

`python evaluate.py`

- Evaluates performance using IoU, precision, recall

6.3 Running Inference

`python predict.py`

- Takes MRI image as input
- Outputs segmentation mask and bounding boxes

7. Running the Web Application

- Step 1: Start Flask Server

`python app.py`

- Step 2: Access Application

Open browser and navigate to:

`http://127.0.0.1:5000/`

8. System Workflow

1. User signs up / logs in
2. Uploads MRI image via drag-and-drop interface
3. Image is validated on client-side
4. Image is sent to backend via REST API
5. Mask R-CNN processes the image
6. Segmentation outputs generated:
 - Bounding boxes
 - Mask overlays
7. Results encoded in base64 format
8. Gemini model generates insights
9. Results displayed on frontend

9. Code

```
from flask import Flask, render_template, request, redirect, url_for, flash, session, jsonify,
send_file
import torch
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
import torchvision.transforms.functional as F
from torchvision.models.detection import maskrcnn_resnet50_fpn_v2
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
from torchvision.models.detection.mask_rcnn import MaskRCNNPredictor
import google.generativeai as genai
import json
import os
import hashlib
from werkzeug.utils import secure_filename
import io
import base64
from datetime import datetime

app = Flask(__name__)
app.secret_key = 'your-secret-key-change-this'

# Upload configuration
UPLOAD_FOLDER = 'static/uploads'
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg'}
app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
os.makedirs(UPLOAD_FOLDER, exist_ok=True)

USER_DATA_FILE = 'users.json'
model = None

# MODEL FUNCTIONS #
def get_model(checkpoint_path):
    num_classes = 2
    model = maskrcnn_resnet50_fpn_v2(weights=None)
    in_features = model.roi_heads.box_predictor.cls_score.in_features
    model.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)
    in_features_mask = model.roi_heads.mask_predictor.conv5_mask.in_channels
    model.roi_heads.mask_predictor = MaskRCNNPredictor(in_features_mask, 256,
num_classes)
    state_dict = torch.load(checkpoint_path, map_location=torch.device('cpu'))
    model.load_state_dict(state_dict)
```

```

    model.eval()
    return model

def transform_image(image):
    return F.to_tensor(image)

def get_prediction(model, image):
    image_tensor = transform_image(image)
    with torch.no_grad():
        prediction = model([image_tensor])
    return prediction[0]

def plot_predictions(image, prediction, threshold=0.5):
    image_np = np.array(image)
    fig, ax = plt.subplots(figsize=(10, 8))
    ax.imshow(image_np)
    scores = prediction['scores'].detach().cpu()
    keep = scores >= threshold

    if keep.sum() == 0:
        return None, "No tumor detected"

    masks = prediction['masks'].detach().cpu()[keep]
    boxes = prediction['boxes'].detach().cpu()[keep]

    for i in range(len(masks)):
        mask = masks[i][0] > 0.5
        ax.imshow(mask, alpha=0.5, cmap='Reds')
        xmin, ymin, xmax, ymax = boxes[i].numpy()
        rect = plt.Rectangle((xmin, ymin), xmax - xmin, ymax - ymin,
                             fill=False, edgecolor="blue", linewidth=2)
        ax.add_patch(rect)
    ax.axis('off')
    buf = io.BytesIO()
    plt.savefig(buf, format='png')
    buf.seek(0)
    img_str = base64.b64encode(buf.getvalue()).decode()
    plt.close(fig)
    return img_str, None

# USER FUNCTIONS #
def load_users():
    if os.path.exists(USER_DATA_FILE):
        with open(USER_DATA_FILE, 'r') as f:

```

```

        return json.load(f)
    return {}

def save_users(users):
    with open(USER_DATA_FILE, 'w') as f:
        json.dump(users, f, indent=2)

def hash_password(password):
    return hashlib.sha256(password.encode()).hexdigest()

def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

# ROUTES #
@app.route('/')
def home():
    if 'username' in session:
        return redirect(url_for('detection'))
    return render_template('home.html')

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        users = load_users()
        username = request.form['username']
        password = request.form['password']
        if username in users and users[username]['password'] == hash_password(password):
            session['username'] = username
            return redirect(url_for('detection'))
        else:
            flash("Invalid credentials")
    return render_template('login.html')

@app.route('/signup', methods=['GET', 'POST'])
def signup():
    if request.method == 'POST':
        users = load_users()
        username = request.form['username']
        password = request.form['password']
        if username in users:
            flash("User exists")
        else:
            users[username] = {
                'password': hash_password(password) }

```

```
    save_users(users)
    session['username'] = username
    return redirect(url_for('detection'))
return render_template('signup.html')
```

```
@app.route('/detection')
def detection():
    if 'username' not in session:
        return redirect(url_for('login'))
    return render_template('detection.html')
```

```
@app.route('/load_model', methods=['POST'])
def load_model():
    global model
    model = get_model("model.pth")
    return jsonify({'success': True})
```

```
@app.route('/predict', methods=['POST'])
def predict():
    global model
    if model is None:
        return jsonify({'error': 'Model not loaded'})
    file = request.files['file']

    if file and allowed_file(file.filename):
        image = Image.open(file).convert("RGB")
        prediction = get_prediction(model, image)
        img_str, error = plot_predictions(image, prediction)
        return jsonify({
            'image': img_str,
            'error': error
        })
    return jsonify({'error': 'Invalid file'})
```

```
@app.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('login'))
if __name__ == '__main__':
    app.run(debug=True)
```

10.. Output Description

- **Bounding Boxes:** Indicate tumor location
- **Segmentation Mask:** Shows tumor shape and boundaries
- **AI Insights:** Provides explanation of tumor characteristics
- **Visualization:** Overlay of segmentation on original MRI image

10. Error Handling & Troubleshooting

- Ensure correct Python version installed
- Check if all dependencies are installed
- Verify dataset path and structure
- Enable GPU for faster training (if available)
- Restart server if Flask app fails to load

11. Notes and Limitations

- System is intended for **educational and research purposes only**
- Not approved for clinical diagnosis
- Performance depends on dataset quality
- Requires moderate computational resources